

Start here

SIH 2026 · SIH26171 — On-device Visual Perception for Light-weight Browser Agents ISRO /
Department of Space · Software · Smart Automation

What we're building, in one paragraph

An AI agent that can see your screen and act on it — **without your personal data ever leaving your machine**. A small model runs inside your browser, reads the page, and strips out every PAN, Aadhaar number, card, password and face *before* anything is sent. The big AI on the server sees only a censored version, works out what to do, and sends back an instruction like "click the Submit button". Your browser does it.

The server does the thinking. It never learns your PAN, your account number, or what you look like.

Why that's hard

The AI is useful only if it can see the screen. But your screen has your salary, your Aadhaar, an open password field. Today you either get a smart assistant that sees everything, or no assistant. We're building the third option.

Platforms

CI runs the suite on Linux, macOS and Windows on every push. On **Windows use WSL** if you can: under Git Bash the server-dependent failure drills cannot run (a background unicorn is spawned but never binds), so they skip with a message. Everything else works.

Get it running

```
git clone https://github.com/AavaranAI/Aavaran.git
cd Aavaran
./setup.sh
```

That takes ~10 minutes, mostly downloads. It checks your tools, fetches the models (~230 MB), builds the extension, sets up the Python server, and runs the tests.

Installing ollama and the model

Only the first step is manual. ollama is a native background service, and neither a shell script nor a browser extension can install one of those for you — that is the one thing you have to do yourself.

1. Install ollama. Pick your platform:

```
# macOS
brew install ollama          # or the .dmg from https://ollama.com/download

# Windows
# OllamaSetup.exe from https://ollama.com/download

# Linux
curl -fsSL https://ollama.com/install.sh | sh
```

On macOS and Windows the app starts itself in the background after installing, so you usually do **not** need to run `ollama serve` by hand.

Check it took:

```
ollama --version
```

2. The model — you have three ways, and you only need one.

	How	When to use it
Automatic	<code>./setup.sh</code>	Default. It already pulls <code>qwen2.5vl:7b</code> if ollama is installed. Nothing else to do.
From the panel	Click Install qwen2.5vl:7b	If setup ran before you installed ollama. Shows a real progress bar.
By hand	<code>ollama pull qwen2.5vl:7b</code>	If you prefer a terminal, or the pull was interrupted.

It is **6 GB**, so on a slow connection start it before you need it.

3. Verify it — with **show**, **not** **list**.

```
ollama show qwen2.5vl:7b
```

`ollama list` displays a model even when its download was interrupted, so it will happily tell you a half-pulled model is present and you will find out at demo time. `show` reads the manifest and the blobs, so it fails when they are incomplete. `setup.sh` checks it this way for exactly that reason. If it fails:

```
ollama rm qwen2.5vl:7b && ollama pull qwen2.5vl:7b
```

Venue wifi: the 6 GB download is the single most likely thing to ruin a demo. Pull it on every laptop in advance. If you need it offline, ollama's blobs live in `~/.ollama/models` and that directory can be copied between machines on a USB stick — worth testing once beforehand rather than discovering it does not work on the day.

None of this is needed for Scan, which is the whole privacy demonstration and runs with nothing installed but the extension.

Running it for real

Three terminals:

```
# 1. the local AI
ollama serve

# 2. our reasoning server
cd server && .venv/bin/uvicorn main:app --port 8975

# 3. a page to test against
python3 -m http.server 8080 --directory bench/pages
```

Then load the extension:

- **Chrome** — `chrome://extensions` → Developer mode → Load unpacked → pick extension/
- **Firefox** — `about:debugging` → This Firefox → Load Temporary Add-on → pick `dist-firefox/manifest.json`. `node build.mjs` produces that folder ready to load; no manifest juggling.

Open `http://localhost:8080/checkout.html`, click our toolbar icon, type a goal, hit Run.

Pre-warm the model first. The first request takes ~17 seconds while 6 GB loads into memory; after that it's ~2.5 s. Never demo on a cold model.

How it's put together

<code>extension/</code>	the browser extension (Chrome + Firefox)
<code>src/content/</code>	reads the page. HOLDS THE VAULT – the only place real values exist
<code>src/pii/</code>	the three-layer PII detector
<code>src/redact/</code>	swaps real values for tags like <code><PII_PAN_1></code>
<code>src/vision/</code>	face detection + image classification, on your GPU
<code>src/agent/</code>	validates whatever the AI sends back, before it touches the page
<code>src/background/</code>	talks to the server. HAS NEVER HELD A REAL VALUE.
<code>src/panel/</code>	the side panel UI
<code>server/</code>	FastAPI + Ollama. Runs locally. No cloud, no API key.
<code>bench/</code>	tests, fixtures, the scorecard
<code>spikes/</code>	the experiments that decided the architecture – read the FINDINGS

The one design idea worth understanding: the vault lives in the content script. The background script is the only part that touches the network, and it has never held a real value. So "no personal data leaves the machine" is a property of the *structure*, not a promise about anyone's code being bug-free. A mistake in the networking code *cannot* leak a PAN, because that code has never seen one.

Where we are

	tuned tests	unseen tests
visual context accuracy	100%	100%
PII detection recall	100%	100%
PII detection precision	100%	100%
redaction precision	100%	100%
data leaks	0	0

Per turn: 47 KB sent, face detection 65 ms, AI ~3.6 s, total ~3.7 s.

Working: the full loop in Chrome and Firefox, face blurring, the privacy ledger, prompt-injection resistance, graceful failure.

Rules we've agreed, and why

1. Never tune against `bench/holdout/`. Those test pages simulate the finale, where the evaluation sites are revealed *on the day*. If we fix a holdout failure by special-casing its content, we've deleted the only evidence we have that this works on pages we haven't seen. The holdout is fully green today — read `bench/README.md` for how its last failure was closed, because the method matters more than the number.

2. No hardcoded CSS selectors, ever. Same reason. Anything tuned to a specific site is worthless at the finale.

3. Measure the operation, not the stage. We've lost hours to confident guesses about what was slow — five wrong guesses across two performance bugs. Instrument first. Both real culprits were invisible from the outside.

4. A green test that can't fail is worse than no test. One verification passed while the extractor was returning *zero nodes*, because `[] .every()` is true. Always assert coverage before correctness.

Useful commands

```
./test-all.sh                # EVERYTHING (no browser needed)
./test-all.sh --full         # + real browsers + live model
node --experimental-strip-types bench/score.ts      # the scorecard
node --experimental-strip-types bench/score.ts --holdout # the unseen set
node --experimental-strip-types bench/leak-test.ts  # the leak invariant (JSON)
node --experimental-strip-types bench/screenshot-leak-test.ts # ...and the image
node --experimental-strip-types bench/injection-test.ts # hostile-page defences
./bench/failure-drills.sh    # things going wrong
node build.mjs               # rebuild after changes
```

Rebuild after any change to `extension/src/` or Chrome will keep running the old code.

Picking something up

Open items are at the bottom of `RESUME.md`. Good first tasks:

- **Grow the test corpus.** We have 14 tuned fixtures, 2 holdout and 4 real sites. Add real-world page shapes to `bench/pages/` with a matching `.truth.json`. This is the highest-value thing anyone can do — every bug we've found came from a new page shape, without exception.
- **Test on a low-end laptop.** Every number above is from an M5. We genuinely don't know what a judge's machine will show, and that gap could matter.
- **UI polish on the side panel.** It's functional, not designed.

Before starting anything, read `RESUME.md` — it lists findings that must not be regressed, each with the reason. Several are non-obvious and were expensive to learn.